FDE ACADEMY · LESSON 0

Building with Claude Code

You steer. Claude types. The tests decide.

Raunak Sharan
Enterprise Agentic AI Engineering

`git checkout lesson-01-start`

OBJECTIVES

By the end of today you can

01

Run the loop: plan, test, implement, verify, review

02

Give Claude context with a CLAUDE.md that keeps clarifications and learnings

03

Use the repo’s commands, skills and lint hook

04

Build a lesson’s modules from its SPEC.md and finish green

05

Triage a review: accept some findings, reject others

06

Say what you must never delegate, and why

WHY THIS WORKS HERE

Generated code is only as good as the check it must pass

Contracts

Typed models at every boundary (lesson 1)

Tests

Both sides of every threshold; 157 of them by lesson 6

Evals

Retrieval eval (lesson 2), golden cases and invariants (lesson 6)

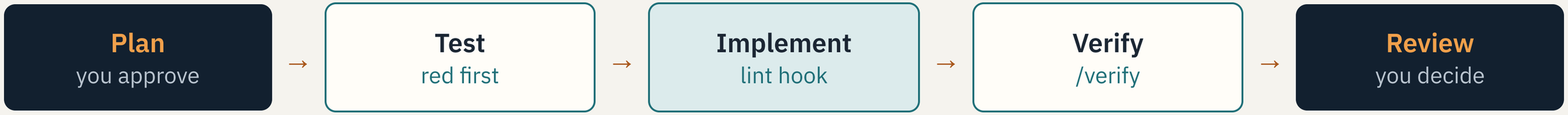
Gates

Zero unsafe approvals, or it doesn’t merge

Claude writes the code. Those checks decide whether it’s right, and you decide what the checks say.

THE LOOP

Five steps, and you own two of them



Step	You	Claude
Plan	Read it, correct it, approve it	Files, tests, failure modes, what it won’t touch
Implement	Watch; interrupt if it wanders	Writes code; fixes lint the hook reports
Review	Accept or reject each finding	A fresh subagent reviews the diff

CLAUDE.MD · COMMITTED, READ EVERY SESSION

The project’s memory, in one file

Context, scope, plan

The business, the ADRs, what’s out of scope, and plan before code.

Six system rules

Engine decides. Never guess. Typed results. No SDK outside llm/. Faults. Audit.

How to work

Karpathy’s four rules: think first, keep it simple, change surgically, end green.

Don’t touch

tests/, evals/, thresholds, golden answers, the corpus.

Clarifications

Questions the spec left open, and the answer that stands. Never re-asked.

Learnings from PRs

One line per lesson from review. It grows every lesson.

THE .CLAUDE/ FOLDER

Repeated work, written down once

File	What it does
settings.json	Pre-approves uv, ruff, pytest, underwriter, read-only git; denies .env
hooks/lint.sh	After every .py edit, runs ruff; problems go straight back to Claude
commands/	/start-lesson · /clarify · /plan-lesson · /build-module · /verify · /review · /learn · /catch-up · /explain-decision
skills/, process	karpathy-guidelines, plus clarify · plan · TDD · debug · verify · review, adapted from Superpowers
skills/, use case	policy rule · tool check · golden case · sample app · regulation · retrieval case · prompt version · workflow node · investigate

STARTER BRANCHES · LESSON-NN-START

Specs and tests given; whole modules left for you

```
# src/underwriter/rag/fusion.py on lesson-02-start
"""Reciprocal rank fusion.

You build this module in this lesson.
Its public API, behaviour and exact
messages are in lessons/02-rag/SPEC.md;
the tests are the answer key.
"""
```

In the workshop

lesson-01-start … lesson-04-start: engine, retrieval, LLM agent, tool agent.

Take-home

lesson-05-start and lesson-06-start: workflow graph, eval harness.

SPEC.md + PROMPTS.md

What to build; then clarify, plan, scaffold, one module per step, verify.

NEVER DELEGATE

Four things stay with you

Policy thresholds

Business commitments, signed off by people accountable for them.

Golden answers

If the system writes the answer key, the eval proves nothing.

Your explanation

The “why it works” paragraph is what the rubric and an interviewer grade.

Review verdicts

A reviewer you can’t overrule is just a second author.

HOW ANTHROPIC DESCRIBES ITS OWN USE

Mostly built-in features, used with discipline

Published practice	In this repo
Plan first, iterate on the plan	/plan-lesson, which waits for approval
Team CLAUDE.md that learns	Clarifications and Learnings from PRs, in git
TDD with check-ins	SPEC.md, red tests first, the tdd-module skill
Verification loops	Lint hook, /verify, the eval gate
Slash commands, hooks, subagents	/verify, /review, lint.sh
Humans review and steer	You approve plans and triage reviews

Sources: “How Anthropic teams use Claude Code” (Anthropic); Boris Cherny’s workflow, InfoQ, Jan 2026

LAB · 70 MINUTES

Lesson 1, from the starter branch

```
git checkout lesson-01-start && uv sync --all-groups
claude
/start-lesson 01
/plan-lesson 01 # read it, correct it, approve it
/build-module src/underwriter/risk/calculations.py
/verify · /review · /learn # triage findings, keep the lessons
```

Break the hook: ask for an unused import; watch it get fixed.

Catch a bad plan: one that edits a test. Send it back.

Record a learning with /learn when Claude gets one wrong.

HOMEWORK AND INTERVIEW PREP

What strong work looks like

Criterion	Strong	Weak
Plan	Corrected, then approved	Approved as generated
Verification	Every check green, shown	“Works for me”
Review	Findings triaged	All or none accepted
Explanation	Your words, specific	Pasted from Claude
CLAUDE.md	Grows with real clarifications and learnings	Untouched

Interview questions

- Use AI tools without shipping code you don’t understand
- What goes in a CLAUDE.md, and how it changes
- Stop an assistant “fixing” a test by editing it
- What you’d never let AI write unreviewed

TAKEAWAY

Let Claude write the code. Keep the judgement, and make it checkable.

Next · Lesson 1

Business problem to AI architecture

Where AI belongs in underwriting, where it must not, and the baseline it has to beat.